Architecting a Baseball Dynasty Simulation: A Comprehensive Conversion Framework from American Football

1. The Paradigmatic Shift in Simulation Architecture

The transition from an American football dynasty simulation to a baseball dynasty simulation requires a fundamental deconstruction and complete rebuilding of the underlying game engine, database architecture, and artificial intelligence logic.[1] Football simulations are inherently collision-based, macro-strategic, and restricted by highly structured weekly schedules featuring synchronized player interactions on a localized physical plane.[1] The core simulation loop of a football game processes all twenty-two players on the field simultaneously, relying heavily on spatial physics, play-calling matrices, and physical attribute checks.[1] Conversely, a baseball simulation is a state-based, discrete-event ecosystem governed by probabilities derived from isolated individual matchups.[5]

To engineer a persistent baseball dynasty game, the architectural foundation must shift from spatial-tactical simulations to stochastic modeling. A baseball game at its atomic level can be perfectly defined by four primary variables: the inning, the number of outs, the configuration of baserunners, and the current score.[5] Because baseball is highly discrete, an overarching simulation engine does not necessarily require complex spatial physics or continuous 3D rendering to be authentic.[2] Instead, authenticity relies on the extreme accuracy of matchup probabilities, the granular application of fatigue curves, and the replication of labyrinthine roster management rules that span deep minor league hierarchies.[6]

The systemic rhythm of the two sports also dictates severe architectural divergences. A football dynasty engine processes a maximum of twenty to twenty-five games per team annually, allowing for manual intervention and linear database expansion.[3] A baseball dynasty engine must comfortably process 162 regular-season games per team, extensive spring training schedules, and a multi-tiered postseason, amounting to thousands of simulated events per season across multiple minor league affiliates.[9] Generating a functional 162-game schedule dynamically is notoriously difficult, often prompting simulation developers to rely on pre-generated external files imported directly into the database.[3] Modern baseball schedules rely on a balanced format, requiring the scheduling algorithm to allocate roughly 32 percent of a team's games against divisional opponents, 28 percent to interleague play, and the remainder to intra-league non-divisional opponents.[9]

Consequently, transitioning a football codebase to baseball requires rewriting the simulation clock, expanding the database schema to handle massive volumetric data, reconstructing player evaluation algorithms, and designing an entirely new progression and attrition model for player development.[12] The market for baseball simulations reveals diverse approaches to solving these exact architectural challenges, ranging from the unparalleled depth of Out of the Park Baseball (OOTP) to browser-based, streamlined experiences like ZenGM Baseball, FranchiseBall, Broken Bat, and WhatIfSports.com's Hardball Dynasty.[13] By analyzing the systems utilized by these established platforms, a comprehensive blueprint emerges for converting a football management framework into a mathematically rigorous baseball dynasty engine.

2. Core Simulation Engine Mechanics and Mathematics

2.1 State-Based Game Logic and the Transition from the Gridiron

In football simulations, continuous clocks and down-and-distance matrices dictate the flow of the game.[4] Translating this to baseball requires the implementation of a state machine that advances only through discrete pitch or play events.[5] Each plate appearance represents a complex interaction that alters the game state. The baseball simulation loop must evaluate the current state, build a probability table based on the specific pitcher-batter matchup, generate a random number against that probability distribution, and update the state accordingly.[18]

This state transition logic eliminates the need for the synchronized multi-actor animations that define football simulations. Instead, the simulation queries the attributes of the pitcher, the batter, the current fatigue level of the pitcher, the handedness platoon splits, and the defensive capabilities of the fielders.[19] Because baseball outcomes are highly sensitive to these localized variables, the simulation engine must be rigorously tuned using advanced mathematical formulas rather than raw attribute comparisons. Within the browser-based simulation FranchiseBall, this exact paradigm is defined as a matchup-based simulation engine where player skills dynamically change based on their performance, isolating the hitter versus pitcher dynamic as the primary driver of the game state.[16]

2.2 Matchup Probabilities: Implementing the Log5 Paradigm

The cornerstone of any sophisticated baseball simulation engine is the algorithm used to determine the outcome of a batter facing a pitcher. Unlike football, where a high overall rating in tackling might simply overpower a lower rating in agility, baseball interactions are relative to the league average. Averaging the attributes of a batter and a pitcher produces statistically inaccurate results; for example, a.350 hitter facing a pitcher who allows a.350 average will yield an outcome much higher than.350, because both players are significantly deviating from the league norm against average competition.[22]

To solve this, baseball simulation engines employ the Log5 method, originally conceptualized by Bill James and Dallas Adams.[23] The standard Log5 formula estimates the probability of Team

A defeating Team B, but it is expanded in simulation engines to calculate the probability of a specific event occurring between a specific batter and pitcher.[23]

The fundamental formula for estimating the probability of an event when a batter interacts with a pitcher is expressed mathematically, incorporating the batter's true probability of the event, the pitcher's probability of allowing the event, and the league average probability for the event.[23] Advanced baseball engines further refine this using an Odds Ratio method or Bayesian hierarchical models to account for smaller sample sizes and team defensive metrics.[5]

A Bayesian four-variable hierarchical log5 model introduces coefficients to account for park factors, weather, and the defensive efficiency rating (DER) of the fielders behind the pitcher.[26] In such a model, the defensive efficiency of the pitching team is expected to be negatively correlated with the probability of a batter reaching base, heavily influencing the engine's final output.[26] Integrating this mathematical foundation ensures that the core gameplay loop produces realistic, verifiable statistical outputs over a 162-game season, stabilizing the variance that often plagues poorly coded sports games.[28]

Another vital mathematical component is the Pythagorean Won-Loss Formula, which dictates that a team's winning percentage is approximately the square of runs scored divided by the sum of the square of runs scored and the square of runs allowed.[24] In modern engine applications, the exponent is often adjusted from 2 to 1.8.[24] These formulas collectively ensure that the simulation engine behaves predictably and aligns with real-world statistical phenomena over massive sample sizes.

2.4 Managing League Totals as Algorithmic Targets

For a simulation to remain authentic across decades of simulated time, the engine must actively manage statistical inflation and deflation. OOTP 25 overhauled its simulation logic to treat "League Totals" as direct targets for the engine. Previously, increasing a league-wide home run target could paradoxically result in fewer home runs, as the engine would internally recalculate players as possessing below-average power relative to the new, higher baseline.

To rectify this in a modernized engine, proportional output mechanics must be implemented. A player with an internal rating matching the exact league average will aim to produce exactly the statistical average defined in the overarching league settings. An auto-calculation function must continuously adjust league modifiers in the background to maintain this delicate statistical balance, counteracting the natural attribute inflation that occurs as computer-generated prospects populate the league over subsequent decades.

3. Redefining Player Attributes and Evaluation Scales

Converting a football database to a baseball database requires tearing down traditional attribute mappings. Football attributes such as tackling, block shedding, or route running have

no direct equivalents, and forcing them into a baseball context breaks the simulation.[30] Furthermore, relying on generic attributes like raw speed or raw strength creates long-term simulation inaccuracies.

3.1 Establishing the Rating Scale

Different simulations employ vastly different rating scales to evaluate player attributes. ZenGM Baseball operates on a traditional 0-100 scale, where a rating of 90 represents an all-time great, 80 indicates an MVP candidate, 60 equates to a solid starter, and anything below 40 signifies a backup or minor league prospect.[14] Hardball Dynasty similarly relies on a 1-100 scale but separates pitching attributes granularly into "Pitch 1" through "Pitch 5," alongside a specific Groundball/Flyball tendency rating where a score of zero indicates a pure flyball pitcher.[15]

In highly complex engines like OOTP, attribute granularity is pushed to its absolute limits. OOTP 25 shifted its internal tracking from a 1-250 scale to an expansive 1-550 scale, allowing for micro-adjustments in player evaluation that remain hidden from the user interface.[13] The standard Major League average is pegged at an internal rating of approximately 400, while elite players reach 500, and a hard maximum is enforced at 550. For minor leagues, the engine scales averages down to roughly 300 for Triple-A and 250 for Double-A, standardizing how ratings interact across different competitive tiers.

3.2 Granular Attribute Mapping and Separation

When mapping attributes, the football concept of agility or speed must be heavily fractured. Speed in a baseball engine must independently govern the ability to hit infield singles, the capacity to stretch doubles into triples, and the proficiency of avoiding double plays. Separately, a "Stealing Aggressiveness" rating must dictate the frequency of steal attempts, cleanly separating pure athletic speed from baserunning intelligence and situational awareness.

Similarly, hitting attributes must be distinctly segregated. OOTP defines contact strictly as a mathematical combination of Batting Average on Balls In Play (BABIP) and Avoid Strikeout ratings. Power is entirely decoupled from the contact calculation, meaning a high-power slugger may possess an incredibly low contact rating if they frequently strike out. This dynamic must be meticulously balanced within the database to prevent statistical anomalies, as high contact and high eye ratings naturally lead to players who aggressively pounce on mistakes within the strike zone.[33]

The simulation platform ZenGM discovered that relying heavily on physical attributes resulted in unrealistic career arcs, prompting the introduction of distinct Offensive IQ (OIQ) and Defensive IQ (DIQ) ratings.[34] These IQ ratings simulate how a player's understanding of defensive positioning, strike zone discipline, and pitch sequencing compensates for physical decline over time, ensuring that aging veterans remain viable contributors rather than instantly becoming unplayable when their physical speed ratings drop.[34]

Table 1: Attribute Conversion Paradigm (Football to Baseball)

Football Attribute Concept	Baseball Simulation Equivalent	Functional Implementation in Engine
Catching / Hands	Contact (BABIP + Avoid K)	Determines the statistical likelihood of putting the ball in play without striking out.
Throw Power	Arm Strength	Governs outfield assists, relay throws, and a catcher's caught-stealing percentages.[35]
Play Recognition	Offensive/Defensive IQ (OIQ/DIQ)	Modifies positioning, pitch recognition, and resilience against standard aging curves.[34]
Speed / Acceleration	Sprint Speed & Steal Aggressiveness	Split into two distinct arrays governing field movement versus base-stealing frequency.
Tackle / Block Shedding	Framing & Blocking (Catchers)	Dictates strike zone manipulation, passed ball prevention, and wild pitch mitigation.
Play Action / Finesse	Pitch Movement & Stuff	Determines a pitcher's ability to induce weak contact or generate swing-and-miss outcomes.

4. Architectural and Database Restructuring

4.1 Database Schema and Volumetric Tracking

The underlying code architecture must be heavily optimized to handle the exponential increase in data tracking. A standard football season tracks roughly 272 total regular-season games. A baseball season tracks 2,430 regular-season games, each containing an average of 300

pitches, exponentially increasing the volume of database writes.[10]

A robust database schema is mandatory. Tables must be thoroughly normalized, creating separate matrices for players, teams, games, player statistics, team statistics, and play-by-play logs.[12] The game table must function as a nexus, utilizing foreign keys linking the home team, away team, and individual player performance arrays.[36] Due to the sheer volume, historical game logs must be compressed or routinely archived to prevent the save file or remote database from bloating to unmanageable sizes, a process critical for maintaining performance across a fifty-year simulated dynasty.[38]

4.2 Tech Stack and Delta Time Simulation

For web-based or lightweight engines, processing the simulation loop requires distinct timing mechanisms.[12] Developers frequently choose a stack consisting of IndexedDB for local database storage, JavaScript for logic execution, and React for the graphical user interface, allowing the simulation to run entirely within the user's browser.[12] Other architectures utilize Python and Flask backends to handle the heavier statistical calculations off-site.[39]

Regardless of the stack, relying on simple setInterval functions for continuous gameplay simulation creates severe stuttering and desynchronization when processing thousands of discrete events across a multi-game daily slate.[40] The engine must instead be built around the concept of delta time. By calculating the exact elapsed time between function calls, the engine ensures that resource generation, statistical updates, and game state changes remain perfectly smooth and mathematically consistent regardless of the host hardware's processing power.[40]

Furthermore, the simulation must accommodate differing pacing models. FranchiseBall, for example, dictates a persistent online simulation where one month equals one full season, simulating exactly three games per day with the first twenty days dedicated to the regular season and the remainder to playoffs.[16] This ongoing, server-side simulation necessitates a robust architecture capable of handling thousands of concurrent users executing live, in-game substitutions and bullpen calls seamlessly.[16]

5. Roster Management and Transaction Logic

Perhaps the most daunting task in converting a football simulation to baseball is programming the roster rules. Football rosters generally consist of a 53-man active squad and a practice squad. Baseball relies on an intricate, highly regulated hierarchical system that governs player movement between the major leagues and multiple minor league affiliates.[7]

5.1 The Active and Expanded Roster Paradigms

The simulation database must track two primary roster states simultaneously for the major league club: the 26-man active roster and the 40-man expanded roster.[7] To participate in a

major league game, a player must exist on the 26-man roster. However, to be placed on the 26-man roster, the player must first be added to the 40-man roster, a process often referred to in the codebase as "selecting a contract".[43]

The 40-man roster serves as a protective shell. It houses the 26 active players, players on the 7-day, 10-day, and 15-day injured lists, the bereavement list, the paternity leave list, and up to 14 minor league players who are protected from being acquired by other organizations.[7] The 40-man roster is highly rigid; adding a player is simple, but removing one requires executing specific, complex transactional logic.[44]

Table 2: Baseball Roster State Architecture

Database Roster Flag	Player Eligibility & Movement Rules	Functional Constraints
Active (26-Man)	Eligible to play in MLB games. Accrues Service Time daily.	Must be on the 40-Man roster. Capped at 26 players (27 for doubleheaders).[6]
Expanded (40-Man)	Protected from Rule 5 Draft. Eligible for immediate MLB call-up.	Requires an Option Year to be demoted to minors. Removing player requires DFA/Waivers.[7]
Minor League Affiliate	Eligible for MiLB games. Does not accrue MLB Service Time.	If off the 40-Man, subject to Rule 5 Draft based on age and professional tenure.[46]
10/15-Day Injured List	Ineligible to play. Accrues MLB Service Time.	Remains on the 40-Man roster. Frees up a 26-Man roster spot.[7]
60-Day Injured List	Ineligible to play for a minimum of 60 days.	Temporarily removed from the 40-Man roster limit, allowing for roster flexibility.[7]

5.2 Waivers, Minor League Options, and Service Time

The engine must enforce strict rules regarding how players are demoted to the minor leagues. The database must track Minor League Options for every player currently occupying a spot on

the 40-man roster.[6] Each player is granted three option years. During an active option year, a player can be moved back and forth between the 26-man roster and the minor leagues virtually at will, subject only to a restriction limiting the player to a maximum of five demotions per season before outright waivers are required.[6] Once an optioned player spends at least 20 days in the minor leagues during a given season, one option year is permanently consumed.[6]

When a player exhausts all three option years, the AI and the user can no longer simply demote them. The simulation must prompt a Designated for Assignment (DFA) state.[6] The player is removed from the 40-man roster and placed on outright waivers, allowing any other team in the league to claim them and assume their current contract.[7] Only if the player successfully clears waivers can they be legally assigned to the minor leagues.[6]

Service time is another critical integer array that must be tracked daily. A player accrues one day of service time for every day spent on the 26-man roster or the major league injured list.[43] Reaching 172 days equates to one full year of service time, which triggers massive financial milestones such as salary arbitration and free agency eligibility.[43] The simulation must precisely calculate these days, as AI managers will aggressively manipulate service time to retain team control over elite prospects, often holding them in the minor leagues just long enough to delay their free agency by a full year. Furthermore, service time tracking governs the "10-and-5 rights" threshold; any player with ten years of major league service time who has spent the last five consecutive seasons with the same team earns the right to veto any proposed trade, functioning as a hardcoded, full no-trade clause.[47]

5.3 The Rule 5 Draft Mechanics

To prevent teams from indefinitely hoarding prospects in the minor leagues, the engine must execute a Rule 5 Draft event every off-season, typically scheduled in December.[46] The programming logic must scan the entire minor league database and flag players based on their signing age and professional tenure. A player signed at age 18 or younger becomes eligible for the Rule 5 draft after five seasons if they are not added to the 40-man roster.[46] A player signed at age 19 or older becomes eligible after four seasons.[46]

During the draft, teams draft in reverse order of the previous season's standings.[46] They may select unprotected players from other organizations for a fixed fee, currently scaled at $100,000 in modern league settings.[46] However, the simulation must attach a persistent conditional flag to the drafted player: they must remain on the drafting team's 26-man active roster for the entirety of the subsequent season, requiring a minimum of 90 days active to satisfy the rule even if injured.[46] If the drafting team attempts to demote the Rule 5 pick, the player must be placed on outright waivers. If they clear waivers, the engine must force an action offering the player back to their original organization for $50,000.[46] Developing the conditional logic to monitor, flag, and enforce these exact parameters is one of the heaviest

structural lifts in transitioning from a football codebase to a viable baseball product.

6. Player Development and Aging Curves

Football players generally enter a professional simulation near their physical peak, developing rapidly and declining steeply based on athleticism. In baseball, players are drafted at ages 18 to 22 and require years of minor league seasoning, featuring deep, slow-burning development curves.[51]

6.1 Systemic Development Algorithms and Projection Formulas

The simulation engine cannot rely on linear progression models. Player potential must fluctuate based on minor league coaching, statistical performance, and hidden personality traits.[54] Historical data suggests that predicting a prospect's peak can be modeled using algorithmic multipliers. In Hardball Dynasty, an effective heuristic for estimating a prospect's final rating involves taking their first-year development points, multiplying by 2.5, and adding it to their initial rating.[52] However, modern engines implement far more dynamic attrition and growth systems.

The engine should track Potential ratings alongside Current ratings.[54] As players age, their Current ratings slowly climb to meet their Potential. However, Potential is not static. A player who experiences a significant boost across multiple hitting profiles in a single season is statistically highly likely to experience another boost, requiring the AI to actively flag and manually manage these rapidly appreciating assets.[54] Conversely, players who stagnate in lower minor league levels will see their Potential ratings decay. To account for injury interruptions, engines must also simulate rehabilitation boosts. In Hardball Dynasty, users discovered that placing slightly injured prospects on the 60-day disabled list resulted in significant artificial bumps to their durability and health stats, a loophole that demonstrates the intricate relationship between roster placement and attribute generation.[52]

6.2 Specialized Training and the Player Development Lab

Leading simulations like OOTP have introduced dual-layer development systems to grant users agency over this slow process.[55] The first layer involves time management sliders, where a user can manually allocate a prospect's training hours.[55] For example, a manager can instruct a batter to sacrifice time spent on plate discipline or contact skills to focus entirely on increasing bat speed and raw power.[55]

The second layer is the offseason Player Development Lab, a complex system simulating a dedicated practice facility.[55] The engine limits access to a specific number of slots per team, ranging from 1 to 30 depending on the configuration. Prospects are assigned to targeted programs such as generating bat speed, refining running mechanics, improving pitcher

velocity, or adding entirely new pitches to their arsenal.[55]

The simulation calculates the success rate of these programs based on a matrix that includes the program's inherent difficulty, the player's age, baseline metrics, work ethic, and coaching staff ratings. The engine rolls for outcomes categorized strictly into four results: Poor (resulting in rating decreases), No Improvement, Successful, or Outstanding (increasing both current and potential ratings). OOTP further gamifies this by utilizing a color-coded progression UI—red indicates poor performance, orange shows frustration, green guarantees a successful outcome, and blue indicates outstanding progression, creating a volatile but realistic progression ecosystem where high-risk developmental assignments can either salvage a struggling prospect or permanently derail their career arc.

6.3 Attrition and Coaching Influence

Coaching quality in a baseball simulation is paramount, often carrying more weight in the code than raw draft position when determining the long-term success of a minor league system.[54] The simulation must link the development attributes of minor league pitching and hitting coaches directly to the percentage chance of a prospect successfully hitting their growth targets. Furthermore, bench coaches invisibly impact the efficiency of positional coaches, creating a systemic web where a highly-rated bench coach elevates the entire developmental baseline of the organization.[35] In simulations like Hardball Dynasty, a manager's personality attributes even impact in-game performance; a manager with a high "argumentative" rating who argues a call successfully provides a temporary ratings boost to their players for that half-inning, though an ejection forces the bench coach to assume control using default, neutral metrics.[56]

7. Pitching Workload, Rotations, and Bullpen Artificial Intelligence

In a football game, stamina recovers almost fully between weekly games. In a baseball game, pitcher stamina is the single most restrictive and complex operational resource. The engine must track the physical fatigue of every pitcher on the active roster daily, applying decay formulas based on pitch counts, innings pitched, and consecutive rest days.[57]

7.1 Starter Fatigue and Rotation Architectures

The starting rotation is the backbone of the defensive simulation. Teams generally utilize a 5-man rotation to navigate the rigorous 162-game schedule, though historical simulations may use 3-man or 4-man rotations, and modern experimentation sometimes pushes it to a 6-man setup to preserve arm health.[59] The simulation must allow the AI to process rotation settings dynamically, choosing between rigid parameters like a "Strict Order" or optimized parameters like "Start Highest Rested Starter," which bypasses the rotation order if an ace is fully rested due

to off-days.[61]

Fatigue calculations require robust mathematical modeling. A pitch-by-pitch fatigue unit (FU) system provides the most accurate simulation. Every pitch thrown accumulates fatigue, but the accumulation is strictly non-linear. Throwing 20 pitches in a single inning generates significantly more immediate stress than throwing 20 pitches across two clean innings.[62] A formula for calculating the fatigue point (FP) analyzes the number of pitches thrown until the pitcher's efficiency degrades past a target threshold, ensuring pitchers lose velocity and command as their pitch count climbs.[62]

If an AI manager utilizes a 4-man rotation in a modern league context without reducing the pitch count limit, the engine must inflict severe penalties on stamina recovery, drastically increasing the probability of career-altering arm injuries and reducing pitch efficiency due to inadequate rest.[59] The engine must calculate cumulative FU across the season, applying rest multipliers. For instance, if a pitcher's last appearance was 2 to 4 days ago, the fatigue multiplier is doubled compared to a pitcher with 5 or more days of rest, simulating the compounding danger of short rest.[57] Radical rotation concepts, such as the SOMA (Shorter Outings, More Appearances) 4-man tandem rotation, require the engine to cap starters at 80 pitches or five innings, necessitating deep bullpens to bridge the gap.[64]

7.2 Bullpen Management and Leverage States

Transitioning the AI logic from football play-calling to baseball bullpen management is exceptionally complex. A football AI decides whether to punt or go for it on fourth down based on field position and clock management.[4] A baseball AI must decide exactly when to remove a starter and which specific relief pitcher to insert based on game leverage, batter handedness, pitch count, and the rest status of the entire bullpen.[66]

The database must define explicit, selectable roles for the bullpen: Closer, Setup, Middle Reliever, Long Reliever, Specialist, and Emergency Starter.[61]

- **Long Relievers** enter games early when a starter collapses, requiring high stamina attributes to absorb multiple innings and protect the rest of the pitching staff from overuse.[68]
- **Specialists** are utilized purely for their high platoon splits against specific handedness, often facing only one or two critical batters to extinguish rallies.[68]
- **Closers and Setup Men** represent the highest "Stuff" and "Control" ratings on the roster, deployed strictly in the 8th and 9th innings to protect narrow leads.[68]

The engine must calculate a Leverage Index (LI) for every game state. In high-leverage situations—such as bases loaded with two outs and a one-run lead in the seventh inning—the AI must bypass middle relievers and immediately trigger a logic loop to fetch the highest-rated available reliever flagged for "High Leverage" situations, regardless of the inning.[69] Modern

simulations like OOTP and MLB The Show have overhauled their AI to eliminate logic gaps that left elite closers sitting in the bullpen while inferior pitchers blew critical games.[69] This includes allowing for advanced bullpen usage, such as "Bullpen Games," where multiple relievers cover the entirety of a game without a traditional starter, a logic sequence that requires the AI to balance the stamina of the entire 13-man pitching staff over a multi-day horizon to ensure arms are available for the following series.[29]

8. Financial Modeling, Salary Arbitration, and AI Trade Logic

The transaction ecosystem in football revolves around hard salary caps, signing bonuses, and unguaranteed contracts. Baseball operates in a realm of guaranteed contracts, luxury taxes, and a highly structured, heavily litigated pre-free agency period known as Salary Arbitration.[47] Building this financial model requires significant programmatic architecture.

8.1 The Mechanics of Salary Arbitration

When a player accrues between three and six years of Major League service time, they become eligible for salary arbitration.[47] Additionally, the top 22 percent of players with between two and three years of service time qualify as "Super Two" players, granting them an extra year of arbitration eligibility and substantially increasing their career earnings.[47]

During this period, the player's salary is no longer set at the league minimum, but it is not yet dictated by the open free market. The simulation must calculate arbitration awards using predictive regressions based on historical statistics.[73] The primary driver for arbitration estimates is Wins Above Replacement (WAR) or RA9-WAR for pitchers.[74]

The relationship between WAR and salary is non-linear and exponential. Algorithmic modeling indicates that a pitcher's expected salary increases by approximately 14 percent per accumulated WAR, while batters see a 13 percent increase per WAR.[73] Because of the exponential nature of this curve, an elite player moving from 5 to 6 WAR will see a significantly larger raw dollar raise than a replacement-level player moving from 0 to 1 WAR.[73]

The algorithm must also include specific "glory statistics" and service statistics.[74] For pitchers, raw Innings Pitched (IP) and Earned Run Average (ERA) act as heavy multipliers. An interaction term multiplying IP by ERA ensures that pitchers who throw a massive volume of innings with sub-7.00 ERAs are compensated highly for their durability and workload management.[74] Furthermore, traditional milestones carry distinct monetary values within the code: a Win is roughly valued at $60,000, a Save at $50,000, and individual Strikeouts at $3,300.[74] For hitters, Home Runs act as massive financial multipliers, baseline valued at $56,000 each, though the code must penalize players via an interaction term (Plate Appearances multiplied by Batting

Average) if they hit homers but possess abysmal overall batting averages.[74]

Table 3: Arbitration Valuation Modifiers

Statistical Category	Player Type	Algorithmic Financial Impact
Wins Above Replacement (WAR)	All Players	Exponential base driver (14% per WAR for pitchers, 13% for batters).[73]
Home Runs (HR)	Batter	+$56,000 per unit, heavily driving offensive compensation.[74]
Pitcher Wins (W)	Pitcher	+$60,000 per unit, rewarding traditional glory statistics.[74]
Saves (SV)	Pitcher (Reliever)	+$50,000 per unit, isolating closer value from middle relievers.[74]
IP x ERA	Pitcher	Interaction term rewarding high volume combined with acceptable run prevention.[74]
PA x AVG	Batter	Interaction term penalizing high-volume batters with exceptionally low batting averages.[74]

8.2 Free Agency and AI Trade Evaluation

Once a player clears six years of service time, they achieve unrestricted Free Agency.[47] The AI routines governing free agent bidding and trades must be completely reprogrammed from their football counterparts.

In trade logic, the AI must weigh Current Year Stats, Previous Year Stats, and underlying Ratings.[76] A robust simulation allows these weights to be adjusted by the user in the backend (e.g., 15% Current Stats, 50% Ratings, 25% Previous Stats, 10% Two Years Prior).[78] This prevents the AI from being exploited. If the AI relies solely on current stats, a user could trade a

low-potential player experiencing a statistically lucky two-month hot streak for an elite prospect.[76] Conversely, heavily weighting ratings ensures the AI recognizes the true value of its minor league assets and prevents superstar rentals from being acquired for nothing but low-tier minor leaguers.[77]

When evaluating pitchers in trades, the AI must inherently penalize "Fragile" injury ratings, heavily discount relief pitchers due to their real-world statistical volatility, and place massive premiums on starting pitchers under team control with deep, four-to-five pitch arsenals.[80] A starter with a 55 overall rating and five unique pitches is mathematically vastly superior to a 55 overall starter with only three pitches, as they can navigate a batting order multiple times without penalty.[80] The AI evaluation array must reflect this nuance to maintain market realism, forcing human users to pay a premium in prospect capital for durable, controllable pitching.[80]

9. Alternative Game Modes and UI Considerations

As the backend architecture stabilizes, the graphical user interface and alternative game modes must be designed to cater to different player demographics. OOTP offers a "Perfect Team" mode, an online card-collection ecosystem completely divorced from the rigors of franchise management. In Perfect Team, complex systems like scouting, injuries, morale, contracts, and minor league development are disabled entirely. Instead, teams compete in online tiers ranging from Rookie to Perfect, utilizing a 26-man active roster drawn from a 175-player reserve pool. The engine simulates 31 league days every real-world day, demonstrating the engine's capacity to handle hyper-accelerated simulation speeds when unburdened by the deeper narrative mechanics of the franchise mode.

For users seeking a streamlined franchise experience, modes like OOTP's "Drive for the Pennant" strip away minor league management and deep financial wrangling, delegating those tasks to the AI Assistant General Manager. The user focuses solely on major league transactions, jump-in moments during critical game situations (such as a ninth-inning save opportunity), and maintaining team momentum via a specialized meter. Integrating these modular gameplay styles ensures the baseball engine remains accessible to casual fans without sacrificing the rigorous, 5000-variable depth required by hardcore simulation enthusiasts.

10. Strategic Synthesis

Converting an American football dynasty simulation into a fully functional baseball simulation is an immense systemic undertaking. It requires replacing collision-based macro-physics with a discrete, Markov-chain probability engine governed by the Log5 mathematical paradigm.

The developer must transition from a weekly simulation loop to a rigid, 162-game daily schedule utilizing delta time logic to ensure processing stability across thousands of database interactions. The roster architecture must be radically expanded from a simple active squad to

a complex 40-man matrix regulated by minor league options, outright waivers, Rule 5 draft eligibility, and intricate service time calculations. Player development systems must shift from immediate physical impact projections to multi-year, highly volatile minor league progressions heavily influenced by coaching attributes and specialized training programs.

Furthermore, the in-game artificial intelligence must be rewritten to handle the highly specific leverage states of bullpen management and pitch-count fatigue tracking. Finally, the financial ecosystem requires the complete integration of non-linear regressions to accurately simulate the salary arbitration process based on WAR, service time, and interaction statistics. By meticulously implementing these architectural frameworks, mathematical algorithms, and database structures, the resulting simulation will shed its gridiron origins and emerge as a statistically rigorous, highly authentic baseball management experience.

Works cited

1. The Designer's Notebook: Designing and Developing Sports Games - Game Developer, accessed February 24, 2026, https://www.gamedeveloper.com/design/the-designer-s-notebook-designing-and-developing-sports-games
2. So, You Want to Build a Baseball Sim? - DraftKick, accessed February 24, 2026, https://draftkick.com/blog/so-you-want-to-build-a-baseball-sim/
3. • View topic - Scheduling Logic - Draft Day Sports, accessed February 24, 2026, https://www.draftdaysports.com/board/viewtopic.php?f=353&t=35419
4. Modeling Football: Combining ML models and Monte Carlo simulations | by Ian Dorward | DraftKings Engineering | Medium, accessed February 24, 2026, https://medium.com/draftkings-engineering/modeling-football-combining-ml-models-and-monte-carlo-simulations-170fd58cbe11
5. 10 Lessons I Learned from Creating a Baseball Simulator | The Hardball Times, accessed February 24, 2026, https://tht.fangraphs.com/10-lessons-i-learned-from-creating-a-baseball-simulator/
6. Minor League Options | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/transactions/minor-league-options
7. 40-man Roster | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/transactions/40-man-roster
8. Building A Player Development System for Your Program - Simple Sabermetrics, accessed February 24, 2026, https://simplesabermetrics.com/blogs/simple-sabermetrics-blog/building-a-player-development-system-for-your-program
9. New Balanced Schedule vs Old Unbalanced Schedule: ~32% of games will be Divisional (~47% before) ~28% of games will be Interleague (~12% before) : r/baseball - Reddit, accessed February 24, 2026, https://www.reddit.com/r/baseball/comments/12v7o7o/new_balanced_schedule_vs_old_unbalanced_schedule/
10. Major League Baseball schedule - Wikipedia, accessed February 24, 2026,

https://en.wikipedia.org/wiki/Major_League_Baseball_schedule
11. balanced vs unbalanced schedule - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?t=146897
12. So you want to write a sports simulation game « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2019/07/so-you-want-to-write-a-sports-sim-game/
13. OOTP Baseball 26 Go! - App Store, accessed February 24, 2026, https://apps.apple.com/id/app/ootp-baseball-26-go/id6466976437
14. ZenGM Baseball Manual « ZenGM Baseball, accessed February 24, 2026, https://zengm.com/baseball/manual/
15. How to Play Hardball Dynasty: Online Baseball Sim Guide | WhatIfSports, accessed February 24, 2026, https://www.whatifsports.com/hbd/pages/main/PlayersGuide.aspx
16. Franchise Ball: The World's Baseball Simulation Game, accessed February 24, 2026, https://www.franchiseball.com/
17. Broken Bat - Online Baseball Management Game, accessed February 24, 2026, https://www.brokenbat.org/
18. How would you implement a sports simulation game? : r/GameDevelopment - Reddit, accessed February 24, 2026, https://www.reddit.com/r/GameDevelopment/comments/kz1617/how_would_you_implement_a_sports_simulation_game/
19. Little Professor Baseball: Mathematics and Statistics of Baseball Simulation - Bob Carpenter, accessed February 24, 2026, https://bob-carpenter.github.io/games/baseball/math.html
20. Comparison of Prediction Methods for Batter-Pitcher Matchups APPROVED BY SUPERVISING COMMITTEE - The University of Texas at Austin, accessed February 24, 2026, https://repositories.lib.utexas.edu/bitstreams/4d173af4-50b8-441e-9234-a06364e8ca6b/download
21. Beyond the Dugout: Diving Deep Into Franchise Ball, the Realistic Baseball Sim Experience, accessed February 24, 2026, http://oreateai.com/blog/beyond-the-dugout-diving-deep-into-franchise-ball-the-realistic-baseball-sim-experience/a51f13807d30651f1c4cb9f3eca3c7fb
22. Any formula to estimate pitcher vs. batter outcomes? : r/Sabermetrics - Reddit, accessed February 24, 2026, https://www.reddit.com/r/Sabermetrics/comments/ici8te/any_formula_to_estimate_pitcher_vs_batter_outcomes/
23. Matchup Probabilities in Major League Baseball - SABR.org, accessed February 24, 2026, https://sabr.org/journal/article/matchup-probabilities-in-major-league-baseball/
24. The James Function - Mathematics - Williams College, accessed February 24, 2026, https://math.williams.edu/the-james-function/
25. How (And Why) We Built A World Series Simulator - Features - Source - OpenNews, accessed February 24, 2026,

https://source.opennews.org/articles/how-and-why-we-built-world-series-simulator/
26. Modeling the probability of a batter/pitcher matchup event: A Bayesian approach - PMC, accessed February 24, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC6192592/
27. Hi guys. I'm working on a daily predictive MLB simulation based on Fangraphs statistics simulation. Thoughts, advice, criticism are all welcome! : r/Sabermetrics - Reddit, accessed February 24, 2026, https://www.reddit.com/r/Sabermetrics/comments/8dicfa/hi_guys_im_working_on_a_daily_predictive_mlb/
28. Realism vs. Playability in Sports Simulation Games : r/gamedesign - Reddit, accessed February 24, 2026, https://www.reddit.com/r/gamedesign/comments/1cdlybl/realism_vs_playability_in_sports_simulation_games/
29. The Impacts of Increasingly Complex Matchup Models on Baseball Win Probability - arXiv, accessed February 24, 2026, https://arxiv.org/html/2511.17733v1
30. Madden 26: How to Edit Players (Player Bio, Rating, Attributes, Appearance & More), accessed February 24, 2026, https://www.youtube.com/watch?v=sbZuK08odd8
31. Best way to make attributes for MLB remake? : r/SuperMegaBaseball - Reddit, accessed February 24, 2026, https://www.reddit.com/r/SuperMegaBaseball/comments/1d0va97/best_way_to_make_attributes_for_mlb_remake/
32. Manual - Baseball GM Seasons - ZenGM, accessed February 24, 2026, http://baseballseasons.zengm.com/manual
33. How important is Eye? A study : r/ZenGMBaseball - Reddit, accessed February 24, 2026, https://www.reddit.com/r/ZenGMBaseball/comments/1gld4sm/how_important_is_eye_a_study/
34. Player ratings and development beta! « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2018/02/player-ratings-and-development-beta/
35. HBD Pro Tips - Hardball Dynasty | WhatIfSports.com, accessed February 24, 2026, https://www.whatifsports.com/forums/Posts.aspx?topicID=545869&page=1
36. DB Schema for simple baseball stat tracking app : r/SQL - Reddit, accessed February 24, 2026, https://www.reddit.com/r/SQL/comments/1w4uqa/db_schema_for_simple_baseball_stat_tracking_app/
37. Database Schema - Baseball | PDF - Scribd, accessed February 24, 2026, https://www.scribd.com/document/341445390/Database-Schema-Baseball
38. Major League Baseball - Designing & Creating a Database - RPubs, accessed February 24, 2026, https://rpubs.com/Chaitra-Rao/803921
39. Bat-Around Baseball Sim A React app featuring play-by-play simulations of baseball games using real player data. - GitHub, accessed February 24, 2026, https://github.com/adamzev/baseball
40. How to make a game loop for your idle game - GitHub Gist, accessed February

24, 2026, https://gist.github.com/HipHopHuman/3e9b4a94b30ac9387d9a99ef2d29eb1a
41. Online Multiplayer JavaScript Game Tutorial - Full Course - YouTube, accessed February 24, 2026, https://www.youtube.com/watch?v=HXquxWtE5vA
42. Noob question on active and 40 man rosters - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?t=125523
43. Newbie to the game and pretty much Baseball management : r/OOTP - Reddit, accessed February 24, 2026, https://www.reddit.com/r/OOTP/comments/1ocbdq3/newbie_to_the_game_and_pretty_much_baseball/
44. 26 Man? 40 Man? DFA? A quick and simple guide to roster limits : r/Dodgers - Reddit, accessed February 24, 2026, https://www.reddit.com/r/Dodgers/comments/1ebnof3/26_man_40_man_dfa_a_quick_and_simple_guide_to/
45. Major League Baseball rosters - Wikipedia, accessed February 24, 2026, https://en.wikipedia.org/wiki/Major_League_Baseball_rosters
46. Rule 5 Draft | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/transactions/rule-5-draft
47. Service Time | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/transactions/service-time
48. Rule 5 Draft Eligibility - the OOTP Manual! - Out of the Park Developments, accessed February 24, 2026, https://manuals.ootpdevelopments.com/index.php?man=ootp16&page=rule_5_draft
49. accessed February 24, 2026, https://www.mlb.com/glossary/transactions/rule-5-draft#:~:text=Clubs%20draft%20in%20reverse%20order,be%20protected%20within%20four%20seasons.
50. The ABCs of the Rule 5 Draft | MiLB.com, accessed February 24, 2026, https://www.milb.com/news/gcs-35561
51. Game Attributes Customization « Customization « ZenGM Baseball, accessed February 24, 2026, https://zengm.com/baseball/manual/customization/game-attributes/
52. Progression tricks & tips - Hardball Dynasty | WhatIfSports.com, accessed February 24, 2026, https://www.whatifsports.com/forums/Posts.aspx?topicID=493208&page=1
53. Youth Baseball Player Development - Velocity Aging Curves -, accessed February 24, 2026, https://www.drivelinebaseball.com/2021/09/youth-baseball-player-development-velocity-aging-curves/
54. Player Development Megathread - Home-grow a contender every time : r/OOTP - Reddit, accessed February 24, 2026, https://www.reddit.com/r/OOTP/comments/1gw31q1/player_development_megathread_homegrow_a/
55. OOTP 25 - OOTP Wiki, accessed February 24, 2026,

https://wiki.ootpdevelopments.com/index.php?title=OOTP_Baseball:Index
56. Hardball Dynasty Help: Glossaries - GM Basics - WhatIfSports, accessed February 24, 2026, https://www.whatifsports.com/hbd/Help/ViewAllHelp.htm
57. An Introduction to Fatigue Units: A New Method for Evaluating Workloads, accessed February 24, 2026, https://fantasy.fangraphs.com/an-introduction-to-fatigue-units-a-new-method-for-evaluating-workloads/
58. Strat-O-Matic Baseball: Basic Pitcher Fatigue - Sports And Dice - WordPress.com, accessed February 24, 2026, https://sportdice.wordpress.com/2019/02/08/strat-o-matic-baseball-basic-pitcher-fatigue/
59. 4 or 5 man rotations - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?p=3108474
60. Pitching Rotations: 4, 5, or 6 : r/mlb - Reddit, accessed February 24, 2026, https://www.reddit.com/r/mlb/comments/1hoe6l2/pitching_rotations_4_5_or_6/
61. Pitching Staff - OOTP Wiki, accessed February 24, 2026, https://wiki.ootpdevelopments.com/index.php?title=OOTP_Baseball:Screens_and_Menus/Team_Menu/Pitching_Staff
62. Design and Analysis of a Pitch Fatigue Detection System for Adaptive Baseball Learning, accessed February 24, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8711585/
63. In-game fatigue for pitchers - SimLeague Baseball | WhatIfSports.com, accessed February 24, 2026, https://www.whatifsports.com/forums/Posts.aspx?topicID=525528&page=1
64. Friday On The Rox: Three-man, Four-man, Five-man, Six-man rotations - DNVR Sports, accessed February 24, 2026, https://thednvr.com/friday-on-the-rox-three-man-four-man-five-man-six-man-rotations/
65. Using a Monte Carlo Simulation to Propose a Radical Four-Man Rotation - FanGraphs, accessed February 24, 2026, https://community.fangraphs.com/using-a-monte-carlo-simulation-to-propose-a-radical-four-man-rotation/
66. Understanding the Bullpen Game: A Unique Strategy in Baseball - Oreate AI Blog, accessed February 24, 2026, https://www.oreateai.com/blog/understanding-the-bullpen-game-a-unique-strategy-in-baseball/9d961c69e76d0c16a2903473470051b0
67. Decision Modeling and Applications to Major League Baseball Pitcher Substitution, accessed February 24, 2026, https://www.researchgate.net/publication/281848546_Decision_Modeling_and_Applications_to_Major_League_Baseball_Pitcher_Substitution
68. Guidelines for setting bullpen roles and usage options - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?t=267444
69. Bullpen logic is a revelation : r/MLBTheShow - Reddit, accessed February 24, 2026, https://www.reddit.com/r/MLBTheShow/comments/1jgkjpi/bullpen_logic_is_a_reve

lation/
70. Bullpen roles : r/OOTP - Reddit, accessed February 24, 2026, https://www.reddit.com/r/OOTP/comments/xjlbqy/bullpen_roles/
71. How exactly does salary arbitration work? : r/OOTP - Reddit, accessed February 24, 2026, https://www.reddit.com/r/OOTP/comments/3go9sx/how_exactly_does_salary_arbitration_work/
72. MLB Salary Arbitration: Basics | Columbia University Society for Baseball Research, accessed February 24, 2026, https://sbr.studentgroups.columbia.edu/news/mlb-salary-arbitration-basics
73. Modeling Salary Arbitration: Introduction - FanGraphs Baseball, accessed February 24, 2026, https://blogs.fangraphs.com/modeling-salary-arbitration-introduction/
74. Modeling Salary Arbitration: Stat Components - FanGraphs Baseball, accessed February 24, 2026, https://blogs.fangraphs.com/modeling-salary-arbitration-stat-components/
75. Team Versus Player? A Study of Baseball Salary Arbitration and the Arbitrator Exchangeability Hypothesis - Program in Quantitative Social Science, accessed February 24, 2026, https://qss.dartmouth.edu/sites/program_quantitative_social_science/files/program_quantitative_social_science/wysiwyg/devan-fink-qss-thesis-2023.pdf
76. What player evaluation settings - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?t=352928&highlight=evaluation
77. Thoughts on trade logic - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?t=362658
78. AI Trade "LOGIC" - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?t=338787
79. Stats vs Ratings : r/OOTP - Reddit, accessed February 24, 2026, https://www.reddit.com/r/OOTP/comments/16n21sc/stats_vs_ratings/
80. Always been curious -- What do most OOTP players consider the trade value of these prospects to be? - Reddit, accessed February 24, 2026, https://www.reddit.com/r/OOTP/comments/187kbe2/always_been_curious_what_do_most_ootp_players/